

Experimenting with Multi-Agent Software Development: Towards a Unified Platform

Malik Abdul Sami*
malik.sami@tuni.fi
Tampere University
Tampere, Finland

Muhammad Waseem Awan*
muhammad.m.waseem@jyu.fi
Jyväskylä University
Jyväskylä, Finland

Zeeshan Rasheed
Tampere University
Tampere, Finland
zeeshan.rasheed@tuni.fi

Mika Saari
Tampere University
Tampere, Finland

Kari Systä
Tampere University
Tampere, Finland
kari.systa@tuni.fi

Pekka Abrahamsson
Tampere University
Tampere, Finland
pekka.abrahamsson@tuni.fi

ABSTRACT

Large language models are redefining software engineering by implementing AI-powered techniques throughout the whole software development process, including requirement gathering, software architecture, code generation, testing, and deployment. However, it is still difficult to develop a cohesive platform that consistently produces the best outcomes across all stages. The objective of this study is to develop a unified platform that utilizes multiple artificial intelligence agents to automate the process of transforming user requirements into well-organized deliverables. These deliverables include user stories, prioritization, and UML sequence diagrams, along with the modular approach to APIs, unit tests, and end-to-end tests. Additionally, the platform will organize tasks, perform security and compliance, and suggest design patterns and improvements for non-functional requirements. We allow users to control and manage each phase according to their preferences. In addition, the platform provides security and compliance checks following European standards and proposes design optimizations. We use multiple models, such as GPT-3.5, GPT-4, and Llama3 to enable to generation of modular code as per user choice. The research also highlights the limitations and future research discussions to overall improve the software development life cycle. The source code for our uniform platform is hosted on GitHub, enabling additional experimentation and supporting both research and practical uses.

KEYWORDS

Software Development, Large Language Models (LLMs), Generative AI, Software Engineering, Compliance, User Requirements, Automated Testing

ACM Reference Format:

Malik Abdul Sami, Muhammad Waseem Awan, Zeeshan Rasheed, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2018. Experimenting with Multi-Agent Software Development: Towards a Unified Platform. In . ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

The Software Development Life Cycle (SDLC) includes methodologies and practices that development teams employ to plan, architect, implement, test, deploy, and maintain software [5]. The first phase, Requirements engineering plays a crucial role in software engineering by linking technical specifications designed by engineers with the ultimate goals of software systems [6]. During all stages of development, including proposal development, design, implementation, and testing, the team regularly reviews and refines these requirements [3, 23]. This ongoing process ensures the software fulfills user needs, which are typically expressed as user stories or tasks. One of the main challenges is the accurate translation of user requirements into technical specifications that accurately meet users' expectations. Software architecture and design become essential once the team defines the requirements clearly and prioritizes them [1, 7]. UML diagrams help developers understand how to implement user stories [2, 24]. This phase emphasizes the importance of meeting requirements and keeping track of changes or mismatches during development.

The development phase in the SDLC focuses on writing code based on user requirements. It is important to maintain thorough documentation to help developers write backend, frontend, or database code. Testing, which includes both unit and end-to-end testing, is another key step in the project. As the need for faster development grows, the field of software engineering is rapidly evolving. It is critical to identify precise and clear requirements necessary for using a test case scenarios suite that effectively validates and verifies software outputs [42, 44].

Artificial Intelligence (AI) and Natural Language Processing (NLP) enable new generation of computational tools that can generate text with near-human accuracy using generative AI [10]. The development of advanced language models, particularly within the Generative Pre-trained Transformer (GPT) series, represents a significant advancement in this area [26, 30, 32]. The recent rise of Large Language Models (LLMs) has started a transformative time in programming and computer program building [52]. Trained on

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

broad datasets related to coding, these LLMs have created a careful understanding of programming, which permits them to exceed expectations in different code-related errands. Examples of such tools include LLaMA and ChatGPT [37, 46].

These LLMs are good at understanding natural language, marking a major shift in software engineering from simple tools to collaborative aids. LLMs provide insights and analysis that were not possible with traditional methods [21, 33, 38]. Recently, multi-agent systems have made significant progress in solving complex problems in software development by emulating development roles [11, 34, 39, 51]. Lin et al. [18] paper introduces a code generation framework inspired by established software engineering practices. Leverages multiple LLM agents to emulate various software process models, namely LCGWaterfall, LCGTDD, and LCGScrum [18]. Tang et al. [43] developed self-collaboration, which assigns LLM agents to work in collaborative software development. Wang et al. [49] proposed an LLM-powered multi-agent collaborative system, named XUAT-Copilot, for automated UAT. While some research has explored integrating multi-agents within LLM frameworks [28, 50, 55], their research focus diverges from the influence of the software process model on software design and code generation of SDLC.

Despite the promising applications of LLMs in automating software engineering tasks, it is important to recognize that software development is a collaborative and teamwork effort [16]. In the real world, developers and stakeholders work together, by following a unified platform that meets all the stakeholders under one umbrella [36]. They improve the process but do not provide a unified platform approach where requirements to user stories, prioritization, design architecture, and both generations of frontend and end-to-end with security, best practices, and patterns according to standards are defined [29].

Using LLMs to convert requirements into user stories, and then to design, generate code, and generate test cases is a necessary step in the ever-changing software development industry. This approach meets the critical need to automate, enhance, and perfect the creation and validation of requirements through acceptance criteria [17, 45, 53].

The **objective** of this research is to develop a comprehensive software development platform that utilizes various LLM-based agents. This platform is designed to convert user requirements into structured software development outputs such as user stories, prioritization, UML diagrams, APIs, unit tests, and end-to-end tests. It will generate code in Python and use frameworks suitable for both backend and frontend components. Additionally, the platform will organize tasks, perform security and compliance, and suggest design patterns and improvements for non-functional requirements. The ultimate goal is to enhance the efficiency, compliance, and quality of software development by integrating generative AI and human involvement in each phase of the software development lifecycle, ensuring the software is ready for deployment.

The **key contributions** of this study are summarized as follows:

- We enhance our existing work by incorporating additional prioritization techniques into a unified platform that generates user stories and prioritizes requirements using large language models (LLMs).
- We have bridged the gap between requirements engineering and technical development by generating UML diagrams with the help of the LLM agent and PlantUML for the software Architect phase.
- We generate a modular Python code with unit tests, and for the client side using React and end-to-end test cases.
- We integrate a multi-AI agent model that utilizes LLMs to automate the SDLC process with user involvement, significantly speeding up the SDLC using GPT-3.5, GPT-4, and llama3.

Structure of the Paper: Section 2 reports the background work related to this study. Section 3 explains the proposed unified platform. Section 4 presents the initial results, and Section 5 discusses the key findings. Section 6 discusses the possible threats and limitations of this study. Section 7 concludes the study.

2 BACKGROUND

In this section, we briefly present the background of the study with a focus on existing research. Section 2.1 provides an overview of studies concerning generative AI and its application across software engineering. Section 2.2 examines works that have utilized LLMs for software development.

2.1 Generative AI in Software Engineering

Generative AI assists in creating outputs that mimic human performance across diverse fields including multimedia production, computer vision, and NLP [14]. The goal is to generate new content. Using autoregressive language models such as GPT and Generative Adversarial Networks (GANs), this technology transforms text production, machine translation, conversational systems, and code generation [4, 50]. The transformer architecture, which significantly enhances NLP capabilities by recognizing contextual relationships in texts, is crucial for the GPT model's ability to produce text that closely resembles human writing [30].

Recent advancements show the versatility and effectiveness of GPT models and their potential to revolutionize software engineering methods [19, 26]. These models enable the automation of several tasks in the SDLC, such as error identification, code snippet creation, and documentation, by utilizing large code databases for training [12, 47]. Integrating GPT models into SE enhances the efficiency and quality of software production, and accelerates coding and application development [8, 20].

The emergence of generative AI in SE marks a significant shift, emphasizing its capability to refine and speed up software development processes. Integrating GPT models into SE heralds a fundamental transformation, impacting how software is crafted and maintained. Importantly, this advancement streamlines the prioritization of requirements—a key phase in development. The application of AI allows developers to generate test suites and test case scenarios more effectively, ensuring that critical features align with user needs and organizational objectives. This approach promises a move towards a more user-centric, and efficient paradigm in software development and testing [48].

2.2 Large Language Model in Software Development

Code generation stands as a crucial research area because it helps reduce development costs. Training more powerful LLMs and exploring prompt-based and agent-based techniques for code generation are currently the most promising research directions.

Agent-based code generation focuses on the importance of defining roles and facilitating communication among multiple LLM agents. Some methods use external tools as agents. One such example is the test executor agent introduced by Huang et al. [15], which provides test logs for LLMs via a Python interpreter. A debugger agent that generates control flow graph information using a static analysis tool is introduced by Zhong et al. [54] to assist LLMs in finding flaws. Several other studies designate LLMs as agents by simulating human jobs, including chief technology officers, analysts, engineers, testers, and project managers. To tackle difficult and innovative challenges in autonomous software development, Zeeshan et al. [34] present a codePori that uses LLM-based multi-AI agents. For example, system design, code development, code review, code verification, and test engineering are all tasks that each agent works on specifically.

Many studies employ GPT models across the SDLC, including requirements gathering, code generation, unit, and GUI testing [31]. LLMs are used to develop a unified platform that potentially makes software development more efficient and effective. However, challenges such as the generation of inaccurate information (e.g., hallucinations) and limitations in natural language comprehension and domain knowledge highlight a significant need for further research [41]. A solution is necessary that can understand diverse client needs, automatically generate user stories, create UML diagrams, and use LLMs to efficiently organize backend and end-to-end test cases [25, 40]. Modern software development methods that prioritize flexibility and intelligent use of LLM agents should be adopted by this tool. By meeting this need, we can improve software quality and fully leverage AI in the SDLC. This approach opens up substantial opportunities for new research and development in the field by integrating an LLM-supported web-based platform, ensuring all critical scenarios are considered and meet user needs.

3 PROPOSED MULTI-AGENT UNIFIED PLATFORM

The architecture of our LLM-based multi-agent model is specifically designed to streamline and enhance the SDLC by incorporating multiple specialized AI agents [35]. Each agent is responsible for handling different aspects of the development process, from requirements engineering to testing. This section outlines the functions and processes of these agents as shown in the provided Figure 1.

3.1 Prompt Engineering

Before exploring each section, we discuss the role of prompt engineering, which is employed in every area of the SDLC. It is considered crucial for generating content. The better the prompt quality, the better the expected results. Below, we have provided the tasks of each agent and their respective work in Table 1.

3.2 Requirements Engineering Agent

The Requirements Engineering agent is an integral part of our engine, leveraging AI capabilities to automate the generation and prioritization of software requirements [40]. This agent begins its process by collecting user requirements. Utilizing APIs from OpenAI and Llama, it interprets and translates these user inputs into structured software requirements. Once generated, the agent employs prioritization algorithms to order these requirements based on factors such as importance, feasibility, and potential impact. The prioritized requirements are then displayed to the requirement engineer and can be downloaded in CSV format for further analysis or documentation purposes.

3.3 Architecting Agent

The Architectural agent assumes responsibility for converting the prioritized requirements into useable design artifacts after requirements engineering. The creation of Unified Modeling Language (UML) diagrams, which provide as a visual depiction of the system architecture and interactions, is the exclusive emphasis of this agent. The agent passes the textual requirements to PlantUML [27] after parsing the structured requirements to determine the required components and their relationships using OpenAI and llama APIs [22]. System designers can then examine and improve the generated UML diagrams, making sure that all system requirements are sufficiently documented during the design phase.

3.4 Code Generation Agent

The Code agent uses AI to automate the coding process throughout the code generation phase, improving frontend and backend development. This agent converts system designs from UML diagrams into executable code with the help of llama API keys and OpenAI. It ensures that the program is both functional and aesthetically pleasing by concentrating on user interface and user experience components in front-end development. For backend development, the agent creates the server-side code needed to manage data, implement application logic, and communicate with databases and other services. After that, the resulting code is shown to the programmers for approval and integration into the overall system.

3.5 Testing Agent

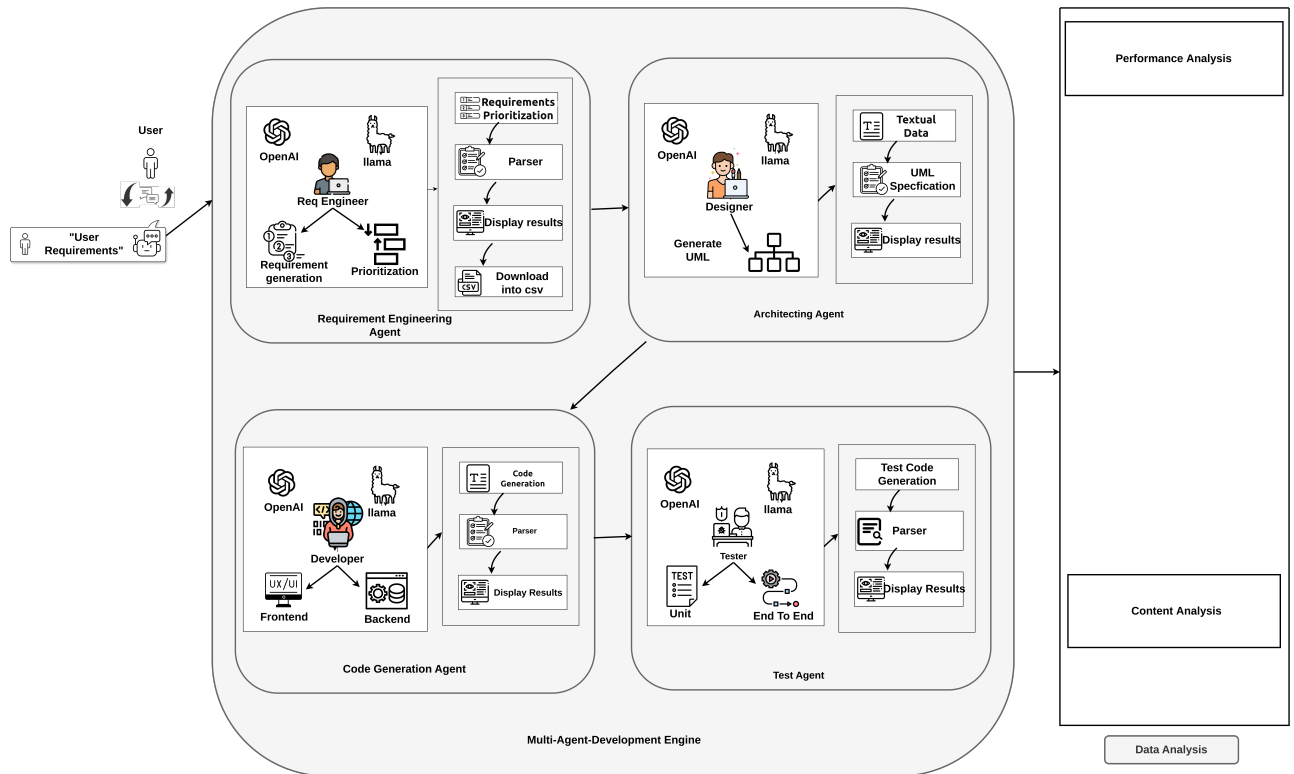
Our engine's testing agent, which is in charge of thoroughly evaluating the software to check its dependability and quality, is the last stage. Unit tests as well as end-to-end tests are generated and run by this app. While end-to-end tests evaluate how different system components interact with one another under simulated real-world settings, unit tests are used to confirm the functionality of particular components. The Testing agent uses llama APIs and OpenAI to automate the creation and execution of test cases. It then presents the findings to testers so they can see any problems and fix them before the program is released.

3.6 Data Analysis

The section includes a strong data analysis to measure the overall effectiveness of the suggested modules. To evaluate the multi-agent system's efficacy in various projects and scenarios, this module has to analyze, how the system is built and what is the performance

Table 1: Prompts and tasks for various agents.

No	Agent	Main Work	Prompt Content
1	RE Agent-01	User Story Generation	Generate user stories from the requirements.
2	RE Agent-02	Prioritization	Prioritized user stories Using AHP, WSJF techniques [40].
3	Architecting Agent	UML Conversion	Act as a senior architect to create PlantUML code from project descriptions, structuring responses with Code and Critique sections.
4	Code Generation Agent	Python and React Code Generation	Act as a senior developer for React and Python, generating clean, optimized code based on project requirements. Structured responses include Response, Code .
5	Test Agent	API and Frontend Testing	Generate unit tests for API and end-to-end tests for frontend applications, ensuring coverage and effectiveness.
6	Compliance Agent	European Standards Compliance	Ensure adherence to European rules, regulations, and security standards, focusing on compliance checks and documentation.

**Figure 1: Developing a unified software development platform through multi-agent collaboration**

of the APIs of each life cycle. We can consistently improve our AI models and development procedures by assessing these factors, which will vov the best possible performance and dependability from the software we produce.

3.7 Technical Background

In this section, we provide the technical details of our proposed unified platform. We proposed 3 algorithms to utilize LLM for SDLC along with the code available on Github [13]. Initially, **Algorithm 01** collected the requirements, which can be supplied either

through direct user input or via files uploaded from the client side. Once the data is entered into the system, the algorithm utilizing an OpenAI-powered LLM—processes and analyzes the requirements. The ultimate output of the algorithm is a set of prioritized user stories, structured to reflect their importance and urgency within the project scope. These user stories are essential for subsequent phases of the SDLC, such as planning and implementation. Additionally, the algorithm provides an option to export these prioritized user stories in a CSV format, facilitating easy integration with other

project management tools and ensuring a seamless transition to later stages of the project development.

Algorithm 1 Requirements Engineering Process with LLM-based agents

Input: Requirements *data* provided via user input or client-side upload.

Output: Prioritized user stories with an option to download as CSV.

```

1: Collect Requirements:
2: User inputs requirements or uploads from the client side (ReactJS).
3: Generate User Stories:
4: objective  $\leftarrow$  Extract objective from data
5: num_stories  $\leftarrow$  Determine number of stories needed
6: user_stories  $\leftarrow$  generate_user_stories_with_epics
7: Prioritize User Stories:
8: for each method in {AHP, MoSCoW, 100 Dollar} do
9:   if method is AHP then
10:    prompt  $\leftarrow$  construct_ahp_prompt(user_stories)
11:   else if method is MoSCoW then
12:    prompt  $\leftarrow$  construct_moscow_prompt(user_stories, method)
13:   else if method is 100 Dollar then
14:    prompt  $\leftarrow$  construct_100_dollar_prompt(user_stories)
15:   end if
16:   prioritized_stories  $\leftarrow$  call_openai_api(prompt)
17:   Parse and store prioritized_stories based on method.
18: end for
19: Download Prioritized Stories:
20: Generate CSV of prioritized user stories based on final prioritization.
21: Provide a download link to the user through a client-side application.
  
```

The process begins with the **Collect Requirements** step, where requirements are gathered either through direct user input or via file uploads from the client side, specifically using ReactJS. Once the requirements are collected, the algorithm transitions to the **Generate User Stories** phase. The main objective is extracted from the data, and the number of necessary user stories is determined. User stories, along with their corresponding epics, are then generated based on the extracted objectives. Following the generation of user stories, Three distinct prioritization techniques are used in the **Prioritize User Stories** phase: the 100 Dollar Test, WSJF, and the Analytic Hierarchy Process (AHP). Appropriate prompts are created using the OpenAI API to make the prioritization process easier, depending on the approach chosen. By using these prompts, it is made sure that user tales are assessed based on their urgency and significance. A refined collection of prioritized user stories that are parsed and saved appropriately is the result of each prioritizing technique.

Downloading a CSV file with the prioritized user tales is the last step, **Download Prioritized tales**. A client-side application is used to make this file available for download, allowing for easy integration into later stages of project management and development. This

method not only automates a large amount of the planning and requirements phases but also makes sure that the prioritization of user stories is both organized and aligned with the project's goals.

Algorithm 2 Software Architecting Agent Process

Input: User stories as input, selection of model and method (e.g., 'gpt-3.5-turbo', 'gpt-4', 'llama3', 'UML-diagram').

Output: Visual UML diagram based on the user stories, displayed on the client side.

```

1: Select Model and Method:
2: Determine the model and method based on user input or system configuration.
3: Prepare API Call:
4: Open the template file corresponding to the method (e.g., "agentplantUML.txt") for the selected model.
5: Replace the placeholder text with an actual project description to form the bot prompt.
6: Add the bot prompt to the conversation history.
7: Call OpenAI API:
8: Configure API headers and post data.
9: Make the API call to the selected model endpoint with the configured data.
10: Handle API Response:
11: if API call is successful then
12:   Extract text and code blocks from the API response.
13:   Separate UML code using specific markers (e.g., "@startuml", "@enduml").
14: else
15:   Raise an exception and output the error.
16: end if
17: Generate UML Diagram:
18: Call the PlantUML open-source API with the extracted UML code.
19: Fetch the UML diagram in byte format.
20: Display UML Diagram:
21: Send the UML diagram to the client-side application for display.
22: Optionally, handle errors and exceptions if the diagram generation fails.
  
```

An automated system for producing UML diagrams based on user stories is described in **Algorithm 02**. The purpose of this technique is to improve the software design phase by automating the generation of architecture diagrams, which are essential for developer instruction and documentation. The first step of the procedure involves choosing the right language model and technique depending on predetermined system parameters or user input. Methods like *UML-diagram* and models like 'get-3.5-turbo', 'gpt-4', and 'llama3' are among the options. After making a decision, the system opens the template file for the selected method to get ready to access the OpenAI API. The project description is then added to this template, which already has placeholder content, to create a thorough bot prompt that is added to the conversation history. An API request is performed to the chosen model endpoint with headers and data that have been defined specifically. The system collects text and code blocks, especially UML code indicated by certain markers

like *@startuml* and *@enduml*, if the API request is successful. The system is built to raise an exception and offer an error output in the case of an API failure. After that, the *PlantUML* open-source API receives the extracted UML code and creates a byte-formatted UML diagram. Ultimately, the client-side application receives this diagram and displays it, providing a clear and graphical depiction of the user narrative. This approach guarantees that complicated UML diagrams are correct representations of the architectural requirements of the project, while simultaneously making the process of creating them simpler.

The purpose of the **Algorithm 03** is to automate the process of creating source code that satisfies particular needs. This is accomplished by using a JSON object that is retrieved from an HTTP POST request. This object holds important information about the model that should be processed, the method to be used, and the content that needs to be generated into code. First, the algorithm takes the requested data and extracts the three key parameters: content, model, and method. After that, the environment is configured using the model and method that have been defined. This entails configuring an authentication key, a related API URL, and a bot prompt file. Then, after loading the bot prompt file, placeholders in it are changed to reflect the material that was supplied in the input.

The system formats the post data to include the model and the customized bot prompt after the environment is suitably established. It then creates authentication headers using the key that was supplied. The required headers and data are then included in a POST request that is delivered to the given URL. The program examines the response it receives from the API call to determine whether the request was successful. Code blocks that are particularly created depending on the input parameters, the chosen model, and the method are extracted and formatted from the payload if the process is successful. The answer then contains these code chunks, which can be used in testing or development settings. If the API call fails, the algorithm manages the issue and produces an appropriate error response. This robust process ensures that developers can obtain tailored code efficiently and reliably, enhancing productivity and facilitating specific development tasks.

4 PRELIMINARY RESULTS

This section reports the preliminary results of the proposed LLM-based multi-agent platform. Our initial findings indicate that the proposed platform is capable of automating the whole process of SDLC with a modular approach. Below, we present the results of our proposed platform in Section 4 along with challenges and possible future agenda.

4.1 Requirement Engineering Phase

We create user stories based on user needs in the requirements engineering area. We apply prioritization techniques like AHP (Analytic Hierarchy Process) and WSJF (Weighted Shortest Job First) in conjunction with LLMs (Large Language Models) to prioritize needs and recommend the best potential values. Algorithm 1 provides additional information about the specifics.

For generating requirements and prioritization, we explore the idea of automating systematic literature reviews (SLR) using large language models and generative AI [38]. We present requirements

Algorithm 3 Modular Code Generator for Development and Testing

Input: JSON object from remote request containing 'content', 'model', and 'method'.

Output: Generated modular code for the specified method and section of development.

```

1: Extract Input: content, model, method ← Parse request data
2: Configure Environment:
3: bot_prompt_file, url, key ← Configure based on model and method
4: bot_prompt ← Load and replace placeholders in bot_prompt_file with content
5: Prepare Request:
6: headers ← Create authentication headers using key
7: post_data ← Format post data with model and bot_prompt
8: API Call:
9: response ← Send POST request to url with headers and post_data
10: status, payload ← Process response checking for success
11: Output Processing:
12: if status = "success" then
13:   code_blocks ← Extract and format code from payload
14:   return Create response with code_blocks
15: else
16:   return Handle error and create error response
17: end if

```

and instruct agents to convert these requirements into user stories and epics. The input we pass into the platform can be seen below:

Snippet of User Requirements

"... The proposed system is designed to streamline research processes by offering a suite of advanced features. With a login feature, it enables users to generate research-specific questions, interact through a React-based user interface, and choose between ChatGPT 3.5 or 4 for language modeling. The system supports API integration, adheres to specific inclusion and exclusion criteria for literature review, and facilitates paper summarization and data extraction.

This algorithm assists in managing software development processes by first collecting user requirements either through direct input or uploads via the client-side application. Once the requirements are collected, the algorithm extracts the main objectives and determines the necessary number of user stories, each associated with broader work themes called epics. Following this, it prioritizes these stories using various methods like AHP (Analytic Hierarchy Process), Moscow (Must have, Should have, Could have, Would like to have), or the 100 Dollar Test, depending on the chosen method. Each method generates a specific prompt that is processed by an AI API to rank the stories in order of importance. The results are then compiled into a CSV file, and a download link is provided for users through the frontend application, allowing them to access the prioritized user stories conveniently. This structured approach ensures efficient planning and execution in software development projects. Below we provided the result of prioritized work in Table 2.

Performance is demonstrated by the AHP-based prioritization process completed in an average of six seconds of generating 10

Table 2: User Stories and Weighted Shortest Job First (WSJF) Prioritization Score [Business Value (BV), Job Size (JS), Risk Reduction (RR), Time Criticality (TC)]

#	User Stories	BV	JS	RR	TC	WSJF
1	As a research student, I want to be able to formulate research-specific questions within the system so that I can define the scope of my study effectively.	7	5	6	4	3.4
2	As a researcher, I need a user-friendly React-based user interface to interact with the system seamlessly and navigate through various features effortlessly.	8	4	5	6	4.75
3	As a language modeling enthusiast, I want the option to choose between ChatGPT 3.5 and 4 for my research tasks to leverage the latest advancements in natural language processing.	6	6	8	7	3.5
4	As a researcher, I require API integration within the system to easily access external data sources and tools for my research projects.	9	7	7	8	3.42
5	As a systematic reviewer, I aim to define precise inclusion and exclusion criteria for literature review within the system to ensure the relevance and quality of the research materials.	8	5	6	5	3.8
6	As a busy researcher, I seek a feature that enables paper summarization and data extraction to save time and efficiently extract key information from research articles.	7	6	7	6	3.33

stories and epic. Weighted Shortest Job First (WSJF) eight to nine seconds to suggest values and prioritize and display results in client-side both techniques showcasing the method's effectiveness and the system's capability to quicken the user story generation and prioritization phases.

In this section, we utilize LLMs to prioritize software requirements using various established methods, including the Analytic Hierarchy Process (AHP), Weighted Shortest Job First (WSJF), and the 100-dollar technique. To the best of our knowledge, this is the first investigation into the use of LLMs for prioritization within requirement engineering. Despite the promising results, several challenges must be addressed to enhance the robustness and applicability of these models. Below, we outline these challenges and propose a future research agenda in this emerging field.

Challenges: While we focus on the state-of-the-art applications of LLMs in requirement engineering, some practical challenges still need to be addressed:

A few of the main challenges facing requirement engineers are the necessity for accurate and unambiguous user stories with well-defined acceptance criteria, the unpredictability of results given the context, the frequent modifications of requirements, and inconsistent output with page refreshes. Subsequent paths are intended to improve requirement engineering efficacy in multiple ways. These include creating a co-pilot feature for automated requirement generation, expanding prioritization techniques to meet a variety of needs, improving document automation for various business documents using advanced AI techniques, and integrating open-source LLMs for cost-effectiveness and performance evaluation.

4.2 Architecting Phase

In the architecture section, we work on software UML diagrams using an AI agent and PlantUML. We use LLM to take requirements from the user and convert them into a specific textual format required by PlantUML, referred to as the PLANTUML response. We then pass this response to the client-side application using the MIME type svg/xml. The details are further explained in Algorithm 2.

This algorithm describes the process of creating visual UML diagrams from user stories with the help of AI models like GPT-3.5 Turbo, GPT-4, or llama3. Initially, the model and method are selected based on user preferences or system settings. The algorithm prepares for an API call by opening a template file specific to the chosen method and filling it with the actual project description, creating a prompt for the AI. This prompt is then sent through an API call to the selected model's endpoint. If the API call is successful, the algorithm extracts UML code from the response, identifies the start and end of the UML block, and sends this code to the PlantUML API, which generates the UML diagram in a byte format. Finally, the diagram is sent to the client-side application for display. If there are any issues during the API call or diagram generation, the system raises an exception and reports the error. This process automates the creation of UML diagrams, ensuring that software architectural designs are accurately visualized based on the initial user stories. Below we provide the result in Figure 2.

In the architecture phase, we use LLM-based agents to convert into textual specifications, which draw a UML diagram however there are some areas where the challenges need to be addressed.

Challenges with using large language models (LLMs) for generating UML specifications include occasional failures in accurate conversions, inconsistent results across different types of diagrams, and variability in outputs upon page refresh. To address these issues, future developments are focusing on integrating a variety of open-source LLMs to enable performance comparisons and broadening the range of supported UML diagrams. Additionally, there are plans to fine-tune these models specifically for creating detailed and

Takeaways

- 1 **Efficient Requirement Prioritization with LLMs:** The integration of Large Language Models with techniques like AHP and WSJF significantly speeds up and enhances the accuracy of requirement prioritization in software development.
- 2 **Associated Challenges and Future Directions:** The use of LLMs in requirement engineering faces challenges like consistency and accuracy, with future research aimed at expanding capabilities and improving the robustness of these technologies.

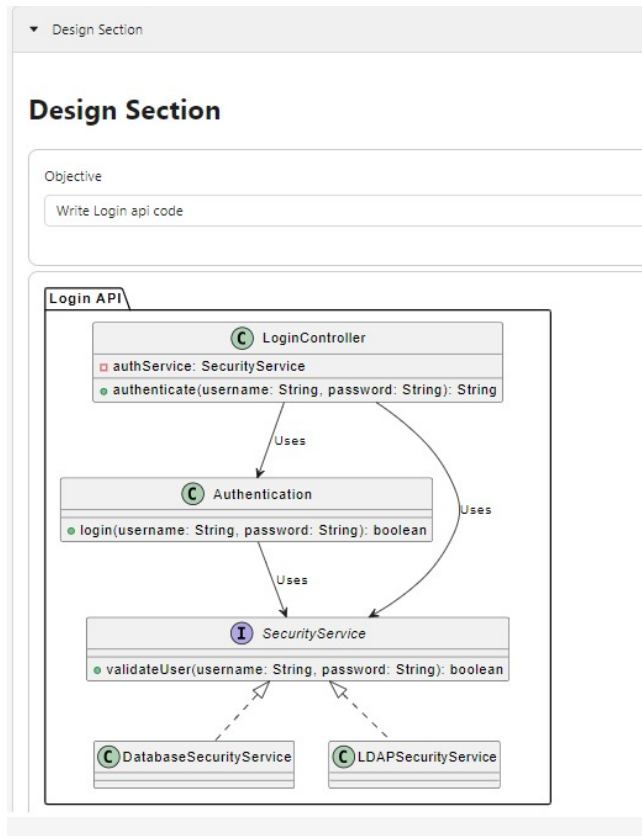


Figure 2: Login Code Design

accurate architect diagrams based on user requirements, enhancing their effectiveness and reliability in software development.

Takeaways

- ③ **AI-Powered UML Diagram Creation:** Utilizing LLM-based agents, the architect process automates the transformation of user stories into textual specifications for PlantUML, enabling the swift generation of accurate UML diagrams displayed via client-side applications.
- ④ **Associated Challenges and Future Directions:** The process faces challenges with occasional failures in accurate UML specification conversion and diagram consistency, prompting future enhancements including exploring diverse open-source LLMs and expanding UML diagram types and model training for optimized architect outputs.

4.3 Code Generation and Test Phase

In this section, we explore four subsections: one for backend code generation, one for frontend code generation, one for generating unit tests for backend APIs, and one for generating test cases for the client side. Our contribution involves generating code for each subsection, including frontend, backend unit tests, or end-to-end scenarios.

The Code Generator for Development and Testing algorithm processes code generation requests efficiently. It starts by receiving a JSON object via an HTTP POST request, which includes the content, model, and method to be used. The algorithm first extracts these inputs from the request data. It then configures the environment by setting up the required parameters such as the file containing the

bot prompt template, the URL to the API, and the authentication key based on the chosen model and method. The bot prompt file is loaded, and placeholders are replaced with the actual content provided. Next, it prepares the API request by creating authentication headers using the key and formatting the POST data with the model and the customized bot prompt. The algorithm then makes the API call, sending the POST request to the specified URL with the prepared headers and POST data. Once the response is received, it checks for success. If successful, it extracts and formats code blocks from the payload and returns them. If not successful, it handles the error and creates an error response. This process enables dynamic generation of code suited for both backend and frontend development based on the specified method, streamlining the development and testing phases.

we present state-of-the-art work that enables code generation for specific parts of development, including backend, frontend, unit testing, and end-to-end testing, using models such as GPT-3.5, GPT-4, and Llama3. Despite the results, many challenges and research directions exist when developing a modular approach to generate test cases and code. An example of the backend code for the login API in Python, along with its unit test, can be seen in the listing 1.

Similarly, the frontend code generated in the React framework for the login feature can also be found in the listing 2

```

1 from flask import Flask, request, jsonify
2 import unittest
3 app = Flask(__name__)
4 users = {
5     "user1": "password1",
6     "user2": "password2"
7 }
8 @app.route('/login', methods=['POST'])
9 def login():
10     data = request.get_json()
11     username = data.get('username')
12     password = data.get('password')
13     if username in users and users[username] == password:
14         return jsonify({'message': 'Login successful'})
15     else:
16         return jsonify({'message': 'Invalid credentials'})
17
18 // Unit test
19 from login_api import LoginAPI
20 class TestLoginAPI(unittest.TestCase):
21     def setUp(self):
22         self.login_api = LoginAPI()
23     def test_login_success(self):
24         response = self.login_api.login(username='
25         test_user', password='password')
26         self.assertEqual(response, 'Login successful')
27     def test_login_incorrect_password(self):
28         response = self.login_api.login(username='
29         test_user', password='wrong_password')
30         self.assertEqual(response, 'Incorrect password')
31     def test_login_invalid_username(self):
32         response = self.login_api.login(username='
33         invalid_user', password='password')
34         self.assertEqual(response, 'Invalid username')
35     def tearDown(self):
36         del self.login_api
  
```

Listing 1: Backend Login API

```

1 // Login.js
  
```



```

2 import React, {useState} from 'react';
3 import axios from 'axios';
4
5 const Login = () => {
6   const [username, setUsername] = useState('');
7   const [password, setPassword] = useState('');
8   const handleLogin = async () => {
9     try {
10       const response = await axios.post('https://api.
11         example.com/login', {
12           username, password
13         });
14       console.log(response.data); // Add logic to handle
15         successful login
16     } catch (error) {
17       console.error('Error logging in: ', error); // Add
18         logic to handle login error
19     }
20   };
21   return (
22     <div>
23       <input type="text" placeholder="Username" value={
24         username} onChange={(e) => setUsername(e.target.
25         value)} />
26       <input type="password" placeholder="Password" value
27         ={password} onChange={(e) => setPassword(e.target.
28         value)} />
29       <button onClick={handleLogin}>Login</button>
30     </div>
31   );
32 };
33
34 export default Login;

```

Listing 2: Frontend Code

Using large language models (LLMs) code generation from requirements include difficulties in understanding context, the need for diverse parsers to differentiate between code and natural language, and a tendency to produce overly generic code. To address these issues, we need to include fine-tuning models for specific development phases, implementing a real-time code review agent, and developing ready-to-deploy software solutions that enhance the effectiveness and efficiency of software development, ensuring robust, cloud-ready deployment solutions.

In examining the challenges and future directions at each stage of the lifecycle, it becomes apparent that certain common challenges need to be addressed across all phases. Addressing these challenges comprehensively can enhance the overall efficiency and success of the lifecycle, paving the way for more robust and sustainable development in future cycles.

Takeaways

- 5 **Automated Code and Test Generation:** An automated system uses LLMs such as GPT-3.5, GPT-4, and Llama3 to dynamically generate backend and frontend code, along with unit and end-to-end tests, based on JSON input and API interactions, streamlining software development.
- 6 **Associated Challenges and Future Directions:** The system faces challenges in context interpretation and requirement ambiguity; future improvements focus on expanding language support and refining AI models to enhance code accuracy and deployment readiness.

5 DISCUSSION

The results from our study emphasize the efficacy of LLMs, particularly GPT-3.5, in redefining software engineering processes across

various stages—from requirement gathering to deployment. Our findings indicate that GPT-3.5 outperforms other evaluated models, such as GPT-4 and Llama3, in tasks related to software development. This superior performance is particularly notable in the platform's ability to translate user requirements into structured software deliverables, including user stories, UML diagrams, API specifications, and comprehensive test suites. The platform also excels in offering best practice advice, performing security and compliance checks, and suggesting software architect and design optimizations. The results indicate that the incorporation of lifecycle management systems (LLMs) into the software development process can yield notable improvements in the end software products' efficiency and quality.

The findings of this study have significant **implications** for the field of software engineering, especially about development process automation and optimization. According to the study, software engineers may depend more on AI to handle repetitive and well-defined tasks, freeing up human resources for more complicated and innovative problem-solving activities. This is because GPT-3.5 effectively automates complex jobs. This change may result in decreased development costs, a shorter time to market, and increased productivity. The fact that our platform complies with European security and compliance standards further suggests that LLMs can be successfully customized to satisfy strict legal requirements, making them a practical option for usage in sectors where data security and privacy are critical. The integration of AI in software engineering also opens up new opportunities for developing more robust, secure, and user-centric software solutions.

Although a number of areas call for more research, the current study offers important insights into the potential of LLMs in software engineering. To begin with, broadening the scope of AI models beyond GPT-3.5, GPT-4, and Llama3 may make it easier to find additional models that are more specialized or efficient for particular software engineering challenges. Subsequent studies may examine how to include further AI technologies, such as machine learning models for decision-making and predictive analytics, into software development cycles. Furthermore, to evaluate the long-term effects of AI integration on software quality, maintenance, and scalability, longitudinal studies are required. In conclusion, given the swift advancement of AI technologies, it will be imperative to consistently assess novel models and revise current ones to sustain the efficacy and significance of AI-powered platforms in software engineering. This ongoing research will help ensure that these AI systems remain adaptive to the changing landscapes of software development requirements and technology standards.

6 THREATS TO VALIDITY

This section reports the potential threats to the validity of this research and its results, along with mitigation strategies that could help minimize the impacts of the outlined threats based on [9]. The threats are broadly classified across external validity, construct validity, internal validity, and reliability.

- **External Validity:** The generalizability of this study may be limited by the application of GPT-3.5, GPT-4, Llama3, and other APIs to automate software development processes in various real-world domains. Differences in training and

model architecture of these models could lead to different outcomes, impacting the practical deployment of AI-driven tools in industries with stringent regulatory requirements such as healthcare and finance. Future research should explore model adaptation across different domains, evaluate integration with diverse technologies, and confirm the practicality of AI in software development through rigorous testing.

- **Construct Validity:** Construct validity concerns how accurately AI models, particularly GPT-3.5 and GPT-4, transform software requirements into functional code and architectural designs. Issues have been observed where AI simplifies or misinterprets complex requirements, potentially compromising the intended functionalities. To mitigate these discrepancies, we have implemented a feedback system at each phase of software development, allowing adjustments to better align theoretical aims with practical outcomes.
- **Internal Validity:** Factors such as project complexity and team expertise with AI can affect the internal validity of our study on multi-agent systems for software development. The stochastic nature of AI models introduces variability, which may lead to misalignments in software development tasks. Addressing these concerns will require rigorous controls, bias mitigation strategies, consistency checks, and comprehensive testing to ensure findings are robust across diverse development scenarios.
- **Reliability:** The reliability of our multi-agent platform is challenged by the inherent variability of LLMs, which can produce different outputs from the same inputs. This variability can impact software functionality and behavior, essential for deployment and maintenance. Strategies to enhance reliability include employing ensemble methods to stabilize model predictions, using strict seed settings to reduce output randomness and further research into model stabilization techniques and consistency checks.

7 CONCLUSIONS

This study proposed a multiagent unified platform that offers a novel investigation into how LLMs might transform the field of SE. Our work has demonstrated notable progress toward automating the conversion from user requirements to fully realized software deliverables through the development and assessment of a unified AI-driven platform. The platform provides a modular approach to testing and deploying software at different phases of the SDLC. It does this by integrating multiple AI agents, including the most advanced models such as GPT-3.5, GPT-4, and Llama3.

According to the results of our manual investigation, GPT-3.5 performs better than its competitors in the majority of the activities, providing a more dependable and effective method for automating software development chores. This superiority highlights how important it is to select models that are suited for given task needs and development circumstances. In addition to expediting the development process, our uniform platform meets security and compliance requirements, particularly those that correspond with European standards. Furthermore, a major value addition that improves the overall robustness and quality of the software solutions is the platform's capacity to provide design improvements and best practices.

A move toward more agile, responsive, and user-focused development approaches is promised by the incorporation of LLMs into software engineering techniques. To fully utilize artificial intelligence (AI) in software engineering, this study lays the groundwork for future developments that will see AI not only assist but also take the lead in the development of novel and dependable software solutions.

There are many intriguing prospects for more research because of the way LLM capabilities are constantly evolving and how they are used in software programming. Software development processes will become even more effective and efficient in areas like enhanced model training methods, model modification for certain SE tasks, and the incorporation of more sophisticated AI tools. Future research will continue to improve these tools by pushing the limits of artificial intelligence in this area, making sure they can satisfy the always-changing and expanding needs of the software business.

8 ACKNOWLEDGMENTS

We express our sincere gratitude to Business Finland for their generous support and funding of our project. Their commitment to fostering innovation and supporting research initiatives has been instrumental in the success of our work.

REFERENCES

- [1] [n. d.]. Weighted Shortest Job First (WSJF) - Scaled Agile Framework. <https://scaledagileframework.com/wsjf/> Accessed: yyyy-mm-dd.
- [2] Pekka Abrahamsson, Outi Salo, Jussi Ronkainen, and Juhani Warsta. 2017. Agile software development methods: Review and analysis. *arXiv preprint arXiv:1709.08439* (2017).
- [3] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. 2023. Towards human-bot collaborative software architecting with chatgpt. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. 279–285.
- [4] Ömer Aydın and Enis Karaarslan. 2023. Is ChatGPT leading generative AI? What is beyond expectations? *What is beyond expectations* (2023).
- [5] Lenz Belzner, Thomas Gabor, and Martin Wirsing. 2023. Large language model assisted software engineering: prospects, challenges, and a case study. In *International Conference on Bridging the Gap between AI and Reality*. Springer, 355–374.
- [6] Elizabeth Bjarnason, Michael Unterkalmsteiner, Markus Borg, and Emelie Engström. 2016. A multi-case study of agile requirements engineering and the use of test cases as requirements. *Information and Software Technology* 77 (2016), 61–79.
- [7] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.
- [8] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration Code Generation via ChatGPT. *arXiv preprint arXiv:2304.07590* (2023).
- [9] Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. 2008. Selecting Empirical Methods for Software Engineering Research. In *Guide to Advanced Empirical Software Engineering*. Springer, 285–311.
- [10] Christof Ebert and Panos Louridas. 2023. Generative AI for software practitioners. *IEEE Software* 40, 4 (2023), 30–38.
- [11] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [12] Yunhe Feng, Sreecharan Vanam, Manasa Cherukupally, Weijian Zheng, Meikang Qiu, and Haihua Chen. 2023. Investigating Code Generation Performance of ChatGPT with Crowdsourcing Social Data. In *Proceedings of the 47th IEEE Computer Software and Applications Conference*. 1–10.
- [13] GPT Laboratory. 2024. GPT Development Tool. <https://github.com/GPT-Laboratory/gpt-developement-tool>.
- [14] Philipp Hacker, Andreas Engel, and Marco Mauer. 2023. Regulating ChatGPT and other large generative AI models. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*. 1112–1123.

- [15] Dong Huang, Qingwen Bu, Jie M Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [16] Vassilka D Kirova, Cyril S Ku, Joseph R Laracy, and Thomas J Marlowe. 2024. Software Engineering Education Must Adapt and Evolve for an LLM Environment. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 666–672.
- [17] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAO-XIANG ZHANG. 2024. SheetCopilot: Bringing Software Productivity to the Next Level through Large Language Models. *Advances in Neural Information Processing Systems* 36 (2024).
- [18] Feng Lin, Dong Jae Kim, et al. 2024. When LLM-based Code Generation Meets the Software Development Process. *arXiv preprint arXiv:2403.15852* (2024).
- [19] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2023. GPT understands, too. *AI Open* (2023).
- [20] Wei Ma, Shangqing Liu, Wenhan Wang, Qiang Hu, Ye Liu, Cen Zhang, Liming Nie, and Yang Liu. 2023. The Scope of ChatGPT in Software Engineering: A Thorough Investigation. *arXiv preprint arXiv:2305.12138* (2023).
- [21] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2023. Experiences from using code explanations generated by large language models in a web software development e-book. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 931–937.
- [22] Meta Platforms, Inc. 2024. Meta Llama. Available at <https://llama.meta.com/>. Accessed: yyyy-mm-dd.
- [23] E. Mnkandla and B. Dwolatzky. 2004. A survey of agile methodologies. *Transactions of the South African Institute of Electrical Engineers* 95, 4 (2004), 236–247.
- [24] Julia Mucha, Andreas Kaufmann, and Dirk Riehle. 2024. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering* (2024), 1–23.
- [25] Anh Nguyen-Duc, Beatriz Cabrero-Daniel, Adam Przybylek, Chetan Arora, Dron Khanna, Tomas Herda, Usman Rafiq, Jorge Melegati, Eduardo Guerra, Kai-Kristian Kemell, et al. 2023. Generative Artificial Intelligence for Software Engineering—A Research Agenda. *arXiv preprint arXiv:2310.18648* (2023).
- [26] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.
- [27] PlantUML Team. 2024. PlantUML. <http://plantuml.com> Accessed: 2024-05-01.
- [28] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
- [29] Ernesto Quevedo, Amr S Abdelfattah, Alejandro Rodriguez, Jorge Yero, and Tomas Cerny. 2024. Evaluating ChatGPT's Proficiency in Understanding and Answering Microservice Architecture Queries Using Source Code Insights. *SN Computer Science* 5, 4 (2024), 422.
- [30] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [31] Asha Rajbhoj, Akanksha Somase, Piyush Kulkarni, and Vinay Kulkarni. 2024. Accelerating Software Development Using Generative AI: ChatGPT Case Study. In *Proceedings of the 17th Innovations in Software Engineering Conference*. 1–11.
- [32] Zeeshan Rasheed, Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Wang Xiaofeng, Anh Nguyen Duc, and Pekka Abrahamsson. 2024. Can Large Language Models Serve as Data Analysts? A Multi-Agent Assisted Approach for Qualitative Data Analysis. *arXiv preprint arXiv:2402.01386* (2024).
- [33] Zeeshan Rasheed, Muhammad Waseem, Kai-Kristian Kemell, Wang Xiaofeng, Anh Nguyen Duc, Kari Systä, and Pekka Abrahamsson. 2023. Autonomous Agents in Software Development: A Vision Paper. *arXiv preprint arXiv:2311.18440* (2023).
- [34] Zeeshan Rasheed, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Codepori: Large scale model for autonomous software development by using multi-agents. *arXiv preprint arXiv:2402.01411* (2024).
- [35] Zeeshan Rasheed, Muhammad Waseem, Kari Systä, and Pekka Abrahamsson. [n. d.]. Large Language Model Evaluation Via Multi AI Agents: Preliminary results. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [36] Martin Rinard. 2024. Software Engineering Research in a World with Generative Artificial Intelligence. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 3–7.
- [37] Konstantinos I Roumeliotis and Nikolaos D Tselikas. 2023. Chatgpt and open-ai models: A preliminary review. *Future Internet* 15, 6 (2023), 192.
- [38] Abdul Malik Sami, Zeeshan Rasheed, Kai-Kristian Kemell, Muhammad Waseem, Terhi Kilamo, Mika Saari, Anh Nguyen Duc, Kari Systä, and Pekka Abrahamsson. 2024. System for systematic literature review using multiple AI agents: Concept and an empirical evaluation. *arXiv preprint arXiv:2403.08399* (2024).
- [39] Malik Abdul Sami, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Tomas Herda, and Pekka Abrahamsson. [n. d.]. Prioritizing Software Requirements Using Large Language Models. ([n. d.]).
- [40] Malik Abdul Sami, Zeeshan Rasheed, Muhammad Waseem, Zheyang Zhang, Tomas Herda, and Pekka Abrahamsson. 2024. Prioritizing Software Requirements Using Large Language Models. *arXiv:2405.01564 [cs.SE]*
- [41] Jaakko Sauvola, Sasu Tarkoma, Mika Klemettinen, Jukka Riekk, and David Doermann. 2024. Future of software development with generative AI. *Automated Software Engineering* 31, 1 (2024), 26.
- [42] Richard Berntsson Svensson and Richard Torkar. 2024. Not all requirements prioritization criteria are equal at all times: A quantitative analysis. *Journal of Systems and Software* 209 (2024), 111909.
- [43] Daniel Tang, Zhenghan Chen, Kisub Kim, Yewei Song, Haoye Tian, Saad Ezzini, Yongfeng Huang, and Jacques Klein Tegawende F Bissyande. 2024. Collaborative agents for software engineering. *arXiv preprint arXiv:2402.02172* (2024).
- [44] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. Chatgpt vs sbst: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering* (2024).
- [45] Archana Tikayat Ray, Bjorn F Cole, Olivia J Pinon Fischer, Anirudh Prabhakara Bhat, Ryan T White, and Dimitri N Mavris. 2023. Agile Methodology for the Standardization of Engineering Requirements Using Large Language Models. *Systems* 11, 7 (2023), 352.
- [46] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [47] Christoph Treude. 2023. Navigating Complexity in Software Engineering: A Prototype for Comparing GPT-n Solutions. *arXiv preprint arXiv:2301.12169* (2023).
- [48] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024).
- [49] Zhitao Wang, Wei Wang, Zirao Li, Long Wang, Can Yi, Xinjie Xu, Luyang Cao, Hanjing Su, Shouzhi Chen, and Jun Zhou. 2024. XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. *arXiv preprint arXiv:2401.02705* (2024).
- [50] Muhammad Waseem, Teerath Das, Aakash Ahmad, Mahdi Fehmideh, Peng Liang, and Tommi Mikkonen. 2023. Using ChatGPT throughout the Software Development Life Cycle by Novice Developers. *arXiv preprint arXiv:2310.13648* (2023).
- [51] Muhammad Waseem, Teerath Das, Teemu Paloniemi, Miika Koivisto, Eeli Räsänen, Manu Setälä, and Tommi Mikkonen. 2023. Artificial Intelligence Procurement Assistant: Enhancing Bid Evaluation. In *International Conference on Software Business*. Springer Nature Switzerland Cham, 108–114.
- [52] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: a ChatGPT-based automated unit test generation tool. *arXiv preprint arXiv:2305.04764* (2023).
- [53] Zheyang Zhang, Maruf Rayhan, Tomas Herda, Manuel Goisauf, and Pekka Abrahamsson. 2024. LLM-based agents for automating the enhancement of user story quality: An early report. *arXiv preprint arXiv:2403.09442* (2024).
- [54] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. LDB: A Large Language Model Debugger via Verifying Runtime Execution Step-by-step. *arXiv preprint arXiv:2402.16906* (2024).
- [55] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406* (2023).